

DEVOPS MASTER COURSE

Volume 9C

Ansible Foundations and Configuration Automation

Chapters 21-30

Inventory, connectivity, ad hoc operations, playbooks, variables, facts, templates, handlers, validation, secrets, and production configuration

2026 Edition

DISCOVER

Inventory and facts

CONVERGE

Idempotent desired state

VERIFY

Check, diff, and evidence

About Volume 9C

Part C begins the Ansible half of Volume 9. Terraform provisions and owns infrastructure resources; Ansible complements it by converging operating-system, middleware, application, and device configuration through agentless automation. This part builds the execution model, inventory discipline, YAML playbooks, data handling, templating, handlers, validation, and secret controls required before enterprise roles and automation platforms are introduced in Part D.

The labs are designed for a local Linux control node and disposable Linux targets such as containers, virtual machines, or localhost. They emphasize idempotency, least privilege, bounded targeting, safe change previews, evidence, and cleanup. Commands are written with Fully Qualified Collection Names where practical to make content provenance clear and reduce module-name ambiguity.

Core principle: Ansible automation is safe only when targeting, identity, variables, and change behavior are explicit. A successful playbook run is not sufficient; production evidence includes bounded host selection, no unexpected changes on the second run, functional validation, secure secret handling, and a recovery path.

Volume 9 Structure and Progress

Part	Primary Coverage	Status
9A	Terraform foundations, state, modules, imports, and team workflow	Complete
9B	Advanced Terraform engineering, testing, policy, Stacks, and enterprise operations	Complete
9C	Ansible foundations, inventory, playbooks, variables, templates, handlers, and configuration automation	This book
9D	Roles, collections, execution environments, Windows/network automation, AWX, and Automation Platform	Next
9E	Terraform plus Ansible delivery pipelines, GitOps, security, compliance, and observability	Planned
9F	Enterprise projects, incidents, certifications, interviews, and integrated capstone	Planned

Part C Chapter Map

Chapter	Focus	Engineering Outcome
21	Ansible operating model, installation, and project configuration	Build a reproducible control-node project.

Chapter	Focus	Engineering Outcome
22	Inventory, groups, host patterns, and dynamic sources	Target the correct systems deterministically.
23	SSH, interpreters, privilege escalation, and connectivity	Establish secure and diagnosable execution paths.
24	Ad hoc commands, modules, return data, and safety	Perform bounded operational actions safely.
25	YAML playbooks, plays, tasks, modules, and idempotency	Express desired configuration as reviewable automation.
26	Variables, precedence, facts, registered data, and filters	Model environment differences without hidden behavior.
27	Conditionals, loops, tags, delegation, and run control	Control execution flow while preserving clarity.
28	Jinja2 templates and configuration-file engineering	Render validated, environment-aware configuration.
29	Handlers, blocks, error handling, check mode, and diff mode	Change services safely and recover from failures.
30	Vault, secret handling, testing, and configuration capstone	Integrate secure, repeatable configuration automation.

Lab Safety and Evidence Standard

- Use disposable targets or localhost for training. Never run learning playbooks against production or employer systems without written authorization and an approved change record.
- Begin every run with an explicit inventory, host limit, and reviewed identity. Avoid broad patterns such as `all` until inventory output and blast radius are verified.
- Use SSH host-key verification, least-privilege accounts, and controlled become rules. Do not disable security controls merely to make a lab pass.
- Prefer idempotent modules over command or shell. When command execution is unavoidable, add `creates`, `removes`, `changed_when`, `failed_when`, `validation`, and `rollback` logic.
- Use `--check` and `--diff` where modules support them, but treat preview as partial evidence rather than a guarantee. Validate again after the real run.
- Protect inventory secrets, Vault passwords, temporary files, verbose logs, rendered configuration, and facts that expose infrastructure details.
- For each lab retain the inventory graph, syntax check, check/diff result, first-run recap, second-run recap, functional test, and cleanup proof.

Chapter 21: Ansible Operating Model, Installation, and Project Configuration

Ansible is agentless from the managed-node perspective, but it still depends on a controlled execution environment, compatible Python and collections, deterministic configuration, and explicit connection behavior. A reproducible control node prevents “works on my laptop” automation failures.

Learning Objectives

- Explain the Ansible control-node and managed-node execution model.
- Distinguish the ansible community package from ansible-core.
- Install Ansible in an isolated Python environment and verify versions.
- Build a project-local ansible.cfg with safe, inspectable defaults.

Core Concepts

Control node: The system that runs ansible, ansible-playbook, inventory plugins, Jinja2 rendering, and most plugins. Production execution should be versioned and reproducible.

Managed node: A target reached through SSH, WinRM, network APIs, or local connection. Traditional Linux automation normally requires no resident Ansible agent.

ansible-core: The automation engine, built-in plugins, command-line tools, and ansible.builtin content. The broader ansible package also installs a curated set of community collections.

Execution environment: A pinned Python or container environment that includes ansible-core, collections, Python dependencies, and system libraries.

Configuration discovery: Ansible can read configuration from environment variables and ansible.cfg locations. The active source must be verified with ansible-config.

Content identity: FQCNs such as ansible.builtin.copy identify the collection and plugin explicitly, improve documentation linking, and avoid naming collisions.

Layer	Responsibility	Production Control
Project	Inventory, playbooks, variables, templates, requirements	Version control and review
Python environment	ansible-core and libraries	Pinned versions and reproducible build
Collections	Modules, plugins, roles	requirements.yml and integrity review
ansible.cfg	Behavior and search paths	Project-local, documented settings
Credentials	SSH keys, tokens, Vault identities	Least privilege and secret manager

Reference Configuration or Command Pattern

```
# Create an isolated control-node environment
python3 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install "ansible-core>=2.19,<2.21"

ansible --version
ansible-config dump --only-changed
ansible-galaxy collection list

# ansible.cfg
[defaults]
inventory = ./inventory
interpreter_python = auto_silent
retry_files_enabled = False
host_key_checking = True
stdout_callback = default

[privilege_escalation]
become_ask_pass = True
```

Production Checklist

- The control-node environment is recreated from documented dependencies.
- The active configuration file and changed settings are recorded.
- Host-key checking remains enabled.
- Collection and Python versions are captured in run evidence.
- Credentials are supplied at runtime and never committed.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
ansible command not found	Virtual environment not active or package missing	Activate environment and inspect <code>python -m pip show ansible-core</code>	<code>ansible --version</code> shows expected executable
Unexpected configuration behavior	Different <code>ansible.cfg</code> or environment variable selected	Run <code>ansible-config view</code> and <code>dump --only-changed</code>	Configuration source matches project
Plugin or module missing	Collection not installed or wrong search path	Inspect requirements and collection paths; install pinned collection	<code>ansible-doc</code> resolves FQCN
Python incompatibility	Unsupported control-node interpreter or dependency conflict	Use supported Python and rebuild clean environment	Version and smoke tests pass

Hands-on Lab 21: Build a Reproducible Ansible Control-Node Project

Prerequisites

- Linux, WSL, or a disposable Linux VM.

- Python 3 and venv support.
- A new Git working directory.

Implementation

1. Create a Python virtual environment and install a pinned ansible-core range.
2. Create ansible.cfg, inventory/, playbooks/, group_vars/, host_vars/, templates/, files/, and requirements.yml.
3. Run ansible --version and record executable, Python, Jinja, and collection paths.
4. Run ansible-config dump --only-changed and verify the project configuration is active.
5. Use ansible-doc ansible.builtin.ping and ansible-doc ansible.builtin.copy to inspect module contracts.
6. Create .gitignore entries for Vault password files, retry artifacts, temporary output, and virtual environments.

Validation Evidence

- A fresh shell can recreate the environment from documentation.
- ansible-config identifies the intended project file.
- ansible-doc resolves built-in modules.
- No credential or local environment artifact is tracked.

Cleanup

- Deactivate the virtual environment.
- Remove the environment only after preserving requirements and evidence.
- Retain the sanitized project skeleton for later chapters.

Interview Questions and Model Answers

1. Why use an isolated Python environment?

It makes Ansible, Jinja, collection, and library versions reproducible and prevents unrelated system packages from silently changing automation behavior.

2. What is the difference between ansible and ansible-core?

ansible-core is the engine and built-in content. The ansible community package includes ansible-core plus a curated collection set.

3. Why use FQCNs?

They identify the content source, reduce collisions, and make module documentation and compatibility easier to audit.

4. Is Ansible completely agentless?

Managed Linux hosts usually need no Ansible agent, but they still require a connection path and typically a compatible Python interpreter for Python-based modules.

Chapter Summary

- Reproducible automation starts with a controlled execution environment.
- Verify configuration and content sources instead of assuming defaults.
- Pin and review dependencies.
- Keep credentials outside code and project files.

Chapter 22: Inventory, Groups, Host Patterns, and Dynamic Sources

Inventory is the blast-radius boundary of Ansible. Correct tasks executed against the wrong hosts are still an incident. Mature inventory design separates identity, topology, connection data, and environment variables while making host selection observable before execution.

Learning Objectives

- Build YAML and INI static inventories.
- Model groups, parent-child relationships, and host/group variables.
- Use host patterns, intersections, exclusions, and `--limit` safely.
- Inspect inventory and understand dynamic-inventory responsibilities.

Core Concepts

Inventory source: A static file, directory, plugin configuration, or executable source that describes hosts and groups.

Group hierarchy: The all group contains every host. Parent and child groups model geography, environment, service, or ownership, but overlapping groups can create variable conflicts.

Host identity: `inventory_hostname` is the stable inventory key. `ansible_host` may specify the actual address and can change independently.

Variable placement: Connection attributes belong near inventory; service policy usually belongs in `group_vars` or role inputs. Avoid embedding secrets in inventory.

Host pattern: Patterns select hosts using groups, unions, intersections, exclusions, and wildcards. Operators and shell quoting require care.

Dynamic inventory: Inventory plugins query authoritative APIs or CMDBs. Production use requires caching, authentication, filtering, ownership, and stale-data handling.

Inventory Concern	Good Pattern	Failure Risk
Identity	Stable logical host name plus explicit <code>ansible_host</code>	Renamed address changes ownership
Environment	Parent/child groups with clear precedence	Host belongs to conflicting groups
Targeting	List hosts before run; use <code>--limit</code>	Broad all pattern changes unintended nodes
Dynamic data	Authoritative source, filters, cache policy	Stale or unfiltered API inventory
Secrets	External secret lookup or runtime injection	Plaintext inventory credentials

Reference Configuration or Command Pattern

```
# inventory/hosts.yml
all:
  children:
    linux:
      children:
        web:
          hosts:
            web01:
              ansible_host: 192.0.2.21
            web02:
              ansible_host: 192.0.2.22
        database:
          hosts:
            db01:
              ansible_host: 192.0.2.31
    production:
      children:
        web: {}
        database: {}

# Inspect before execution
ansible-inventory -i inventory/hosts.yml --graph
ansible-inventory -i inventory/hosts.yml --host web01
ansible 'web:&production:!web02' --list-hosts
```

Production Checklist

- Inventory has an owner, source of truth, refresh policy, and naming standard.
- Every production run records the resolved host list.
- Environment and service groups are not overloaded with unrelated meanings.
- Variables have documented placement and conflict rules.
- Dynamic sources use filtering and fail safely when the source is unavailable.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
No hosts matched	Pattern, inventory path, group name, or quoting error	Run <code>--list-hosts</code> and <code>inventory --graph</code> ; inspect active inventory	Expected hosts are listed exactly
Wrong variable selected	Overlapping groups or precedence conflict	Inspect <code>--host</code> output and move variable to a single policy location	Resolved host variables are deterministic
Duplicate host entries	Multiple sources use different aliases for one endpoint	Adopt stable identity and deduplicate sources	Inventory maps one managed identity intentionally
Dynamic inventory is empty	Credentials, plugin config, filter, or API failure	Run inventory plugin with verbosity and validate source independently	Expected filtered hosts appear

Hands-on Lab 22: Design and Verify a Multi-Environment Inventory

Prerequisites

- The project from Chapter 21.
- Three disposable targets or localhost aliases.
- No real production addresses.

Implementation

1. Create a YAML inventory with development, staging, production, web, and database groups.
2. Add host_vars and group_vars that demonstrate geography and environment values without secrets.
3. Run `ansible-inventory --graph` and `--host` for each logical host.
4. Test union, intersection, exclusion, wildcard, and `--limit` patterns using `--list-hosts` only.
5. Create a second inventory source and observe merge order; then remove ambiguity.
6. Write an inventory runbook defining naming, ownership, variable placement, and stale-source behavior.

Validation Evidence

- Every pattern resolves to the intended host set.
- No host receives conflicting environment identity.
- Resolved variables are documented.
- The runbook includes a mandatory pre-run host-list check.

Cleanup

- Remove test-only inventory aliases.
- Retain sanitized inventory examples and pattern evidence.
- Confirm no secret or real internal address is committed.

Interview Questions and Model Answers

1. Why is inventory a safety control?

It defines which systems exist for Ansible and how host patterns resolve, so errors directly determine blast radius.

2. What is the difference between `inventory_hostname` and `ansible_host`?

`inventory_hostname` is the logical inventory key; `ansible_host` is the connection address or name.

3. How should dynamic inventory be validated?

Inspect the graph and host variables, test filters, define caching and stale-data behavior, and list resolved hosts before execution.

4. Where should variables be placed?

Place values according to scope and ownership, define each policy value once where practical, and avoid secrets in plaintext inventory.

Chapter Summary

- Inventory design is targeting architecture.
- Always inspect the resolved host set before change.
- Keep host identity stable and connection addresses explicit.
- Dynamic sources require filtering, caching, and failure policies.

Chapter 23: SSH, Interpreters, Privilege Escalation, and Connectivity

Connectivity failures are frequently misdiagnosed as playbook problems. Reliable Ansible operations require a known path from control node to target, verified host identity, a compatible remote interpreter, least-privilege login, and tightly scoped elevation.

Learning Objectives

- Configure SSH keys, users, ports, and bastion paths.
- Understand Python interpreter discovery and raw bootstrap operations.
- Use become securely and distinguish connection identity from escalation identity.
- Troubleshoot unreachable hosts systematically.

Core Concepts

Connection plugin: Defines how Ansible reaches the target. SSH is typical for Linux, but local, container, network_cli, httpapi, and WinRM serve other platforms.

Host verification: SSH host-key checking protects against impersonation. Known-host lifecycle must be managed rather than disabled globally.

Remote interpreter: Most Python modules execute through a discovered Python interpreter. Minimal images may require a raw bootstrap before standard modules work.

Connection identity: ansible_user and SSH credentials determine the initial login account.

Privilege escalation: become changes effective privilege for selected tasks or plays. It should use controlled sudo rules rather than unrestricted root credentials.

Bastion path: ProxyJump or ssh_common_args can route through a managed jump host. The path, keys, logging, and failure behavior must be tested.

Layer	Diagnostic	Safe Control
DNS/routing	ssh -vv and reachability tests	Approved network path
SSH trust	Known-host fingerprint verification	Managed known_hosts
Authentication	Key and account test	Short-lived or protected key
Python	ansible_python_interpreter and raw test	Supported interpreter
Privilege	sudo -l and become test	Command-scoped elevation

Reference Configuration or Command Pattern

```
# inventory connection variables
all:
  vars:
    ansible_user: automation
    ansible_python_interpreter: /usr/bin/python3
    ansible_ssh_common_args: >-
      -o ProxyJump=automation@bastion.example.net

# Connectivity diagnostics
ansible all -m ansible.builtin.ping -vv
ansible all -m ansible.builtin.command -a 'id' --become

# Bootstrap only when Python is absent
ansible new_hosts -m ansible.builtin.raw -a 'test -x /usr/bin/python3 || sudo apt-get update && sudo apt-get install -y python3'
```

Production Checklist

- Host-key checking is enabled and fingerprints are obtained through a trusted process.
- The automation login account has no unnecessary interactive or administrative access.
- Become is limited to required commands or roles.
- The expected Python interpreter is documented and validated.
- Bastion and proxy behavior is included in connectivity evidence.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
UNREACHABLE via SSH	Routing, DNS, firewall, key, account, or host-key issue	Reproduce with <code>ssh -vv</code> ; inspect one layer at a time	Manual and Ansible connection succeed
Module fails before execution	Python missing or interpreter wrong	Use raw for controlled bootstrap; set interpreter explicitly if needed	<code>ansible.builtin.ping</code> succeeds
Missing sudo password	Become method requires authentication	Use approved prompt, credential plugin, or scoped passwordless rule	Only intended task elevates
Works directly, fails through bastion	Proxy option, key forwarding, DNS, or route issue	Test exact SSH ProxyJump command and inventory variable quoting	Ansible reaches target through approved path

Hands-on Lab 23: Establish a Least-Privilege Managed-Node Connection

Prerequisites

- One disposable Linux target.
- A non-root automation account.
- SSH key material protected with correct filesystem permissions.

Implementation

1. Create or use a non-root automation account and install its public key.

2. Add the host fingerprint to a controlled known_hosts file.
3. Configure inventory connection variables without embedding the private key or password.
4. Run `ansible.builtin.ping` and gather a minimal fact subset.
5. Create a scoped sudo rule for one harmless administrative command and test it with `--become`.
6. Document diagnostics for DNS, routing, SSH trust, authentication, Python, and become.

Validation Evidence

- Direct SSH and `ansible.builtin.ping` succeed.
- The normal account cannot perform unrestricted root actions.
- The approved become command succeeds and an unapproved one fails.
- The exact interpreter and host fingerprint are recorded.

Cleanup

- Remove the lab sudo rule and temporary account if appropriate.
- Revoke temporary keys.
- Retain sanitized connectivity and troubleshooting evidence.

Interview Questions and Model Answers

1. What does `ansible.builtin.ping` test?

It validates that Ansible can connect and execute a small Python module; it is not an ICMP ping test.

2. Why not disable host-key checking?

It removes protection against host impersonation and turns a trust problem into silent acceptance.

3. When is raw useful?

For bootstrap operations before Python or another required runtime exists; it should be narrowly controlled because it bypasses normal module idempotency.

4. What is the difference between `remote_user` and `become_user`?

The remote user establishes the connection. The become user is the identity used after privilege escalation for selected automation.

Chapter Summary

- Connectivity is a layered system, not a single Ansible setting.
- Protect host identity and credentials.
- Use least privilege for login and escalation.
- Bootstrap with raw only when normal modules cannot run.

Chapter 24: Ad Hoc Commands, Modules, Return Data, and Operational Safety

Ad hoc commands are powerful for discovery and bounded operations, but their speed can bypass review and repeatability. They should be used for diagnosis, emergency containment, and one-time verified actions, then converted into playbooks when a control must be repeatable.

Learning Objectives

- Use ansible ad hoc syntax and host limits safely.
- Select idempotent modules instead of command execution.
- Interpret changed, failed, unreachable, skipped, and return data.
- Control parallelism, privilege, and evidence for one-time operations.

Core Concepts

Ad hoc command: A one-task invocation using ansible rather than a playbook. It still resolves inventory, variables, connection settings, and privilege rules.

Module contract: Each module defines arguments, return values, platform support, check mode, and idempotency behavior. ansible-doc is the authoritative local reference.

FQCN: The full collection-qualified module name makes provenance and documentation explicit.

Return data: Module output commonly includes changed, failed, rc, stdout, stderr, msg, and module-specific fields. Machine parsing should use JSON-capable callbacks or registered data in playbooks.

Forks and serial impact: Ad hoc operations may run across many hosts concurrently. Limit and fork settings control load but do not replace application-aware rollout design.

Command escape hatch: command and shell are not desired-state modules. Use them only with explicit change/failure rules and guards.

Task Type	Preferred Module	Reason
File ownership	ansible.builtin.file	Idempotent state and metadata
Package	ansible.builtin.package	Abstract package state
File copy	ansible.builtin.copy	Checksum and diff support
Service	ansible.builtin.systemd_service	State-aware unit management
Arbitrary program	ansible.builtin.command	Use only with guards and explicit semantics

Reference Configuration or Command Pattern

```
# Inspect exact targets first
ansible web --list-hosts

# Read-only discovery
ansible web -m ansible.builtin.command -a 'uname -r' --check
ansible web -m ansible.builtin.setup -a 'filter=ansible_distribution*'

# Idempotent operation
ansible web -m ansible.builtin.file -a 'path=/tmp/ansible-lab state=directory mode=0750'

# Bounded execution
ansible web --limit web01 --forks 1 -m ansible.builtin.copy -a 'src=files/banner.txt dest=/tmp/banner.txt mode=0644' --diff
```

Production Checklist

- The resolved host list and limit are recorded before execution.
- Read-only discovery precedes change.
- The chosen module documents check-mode and diff-mode behavior.
- The operation is repeatable or has explicit one-time guards.
- The second run proves expected idempotency.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
All hosts report changed every run	Non-idempotent module or volatile input	Use desired-state module; inspect timestamps/templates; set semantics explicitly	Second run reports changed=0
Command succeeds on some hosts only	Platform or prerequisite variation	Inspect facts and module requirements; split target groups	Supported hosts converge and unsupported hosts are excluded
Ad hoc command hits too many hosts	Pattern or limit error	Stop; list hosts; reduce pattern and forks	Only approved targets execute
Output is hard to interpret	High concurrency or unstructured command data	Use a playbook with register/assert, or structured callback	Results are machine-checkable

Hands-on Lab 24: Perform a Bounded Ad Hoc Configuration Operation

Prerequisites

- Two disposable targets or localhost aliases.
- A small text file in files/.
- Inventory verified with --graph.

Implementation

1. List the web host group and save the resolved host list.
2. Run ansible.builtin.setup with a narrow filter to collect operating-system facts.
3. Create a directory with ansible.builtin.file on one limited host.

4. Copy a banner file with `ansible.builtin.copy` and `--diff`.
5. Run the same operations again and verify no change.
6. Remove the file and directory using `idempotent state=absent` operations.

Validation Evidence

- The target list contains only approved hosts.
- First run reports the expected create/change actions.
- Second run reports `changed=0`.
- Cleanup is verified on each target.

Cleanup

- Ensure all created paths are absent.
- Remove temporary evidence containing internal host details.
- Retain sanitized command and recap records.

Interview Questions and Model Answers

1. When are ad hoc commands appropriate?

For discovery, bounded one-time operations, emergency actions, and experimentation. Repeatable controls should be moved into reviewed playbooks or roles.

2. Why prefer modules over shell commands?

Modules expose desired state, structured return data, check/diff support, and platform-aware idempotency.

3. What does `changed` mean?

The module reports that it altered or predicts alteration of managed state; it is not automatically proof that the service is functionally correct.

4. How do you reduce ad hoc blast radius?

List hosts, use explicit inventory and `--limit`, lower forks, start read-only, and run against a canary before a wider target set.

Chapter Summary

- Ad hoc speed must not bypass targeting and evidence controls.
- Use FQCN modules with desired-state semantics.
- Repeat the operation to prove idempotency.
- Convert recurring actions into playbooks.

Chapter 25: YAML Playbooks, Plays, Tasks, Modules, and Idempotency

Playbooks turn operational intent into version-controlled automation. Their quality depends on readable YAML, explicit host and privilege scope, state-aware modules, meaningful task names, deterministic ordering, and acceptance tests beyond the play recap.

Learning Objectives

- Create plays and tasks with valid YAML and explicit scope.
- Use module parameters to express desired state.
- Understand `changed_when`, `failed_when`, and idempotency risks.
- Run syntax, check, diff, first-run, and second-run validation.

Core Concepts

Play: Maps a host pattern to variables, privilege settings, collections, strategy, and an ordered task list.

Task: Invokes a module or action with a descriptive name. Task names should communicate desired outcome, not implementation trivia.

Desired state: A module compares current and requested state, changing only when necessary. Idempotency depends on module behavior and stable inputs.

YAML discipline: Indentation, quoting, booleans, lists, dictionaries, and multiline strings affect parsing. Linting and syntax checks prevent subtle defects.

Change semantics: `changed_when` and `failed_when` customize interpretation but must reflect reality rather than hide noisy or failing tasks.

Acceptance: A recap with `failed=0` proves execution completed; application, security, and service tests prove the desired outcome.

Validation Stage	Command	What It Proves
Parse	<code>ansible-playbook site.yml --syntax-check</code>	YAML and playbook structure
Target	<code>ansible-playbook site.yml --list-hosts</code>	Resolved host scope
Preview	<code>ansible-playbook site.yml --check --diff</code>	Supported predicted changes
Converge	<code>ansible-playbook site.yml</code>	Real execution
Idempotency	Run again	Stable desired state

Reference Configuration or Command Pattern

```

---
- name: Configure a small web service baseline
  hosts: web
  become: true
  gather_facts: true

  tasks:
    - name: Ensure web package is present
      ansible.builtin.package:
        name: "{{ web_package }}"
        state: present

    - name: Ensure document root exists
      ansible.builtin.file:
        path: "{{ web_root }}"
        state: directory
        owner: root
        group: root
        mode: '0755'

    - name: Publish health page
      ansible.builtin.copy:
        dest: "{{ web_root }}/health.txt"
        content: "healthy"
      register: health_status

    - name: Ensure service is started and enabled
      ansible.builtin.systemd_service:
        name: "{{ web_service }}"
        state: started
        enabled: true

```

Production Checklist

- Every play has an explicit host pattern and privilege scope.
- Tasks use FQCN modules and descriptive outcome-focused names.
- File modes are quoted to avoid YAML numeric ambiguity.
- Command/shell tasks have guards and honest changed/failed rules.
- The second run and a service-level test are mandatory evidence.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Syntax check passes but run fails	Remote prerequisites, variables, permissions, or module behavior	Use --start-at-task, -vv, and targeted reproduction	Failure cause is isolated and corrected
Every run changes file	Rendered content or metadata	Remove timestamps/random	Second run is unchanged

Symptom	Likely Cause	Response	Validation
	is volatile	data; inspect diff	
changed_when false hides real change	Incorrect customization	Remove override; model state with module or exact condition	Recap reflects actual modification
Play targets unintended group	Ambiguous pattern or inventory merge	List hosts and use explicit limit	Only approved hosts appear

Hands-on Lab 25: Write an Idempotent Baseline Playbook

Prerequisites

- Disposable Linux target.
- Package and service names appropriate to the target OS.
- Inventory and connection tested.

Implementation

1. Create group variables for package, service, and document-root paths.
2. Write a playbook that installs the package, creates the directory, writes a health file, and starts the service.
3. Run `--syntax-check`, `--list-hosts`, and `--check --diff`.
4. Apply to one canary host and run a functional health test.
5. Run the playbook again and require `changed=0`.
6. Introduce drift in the health file, preview the repair, apply, and validate.

Validation Evidence

- Syntax and target checks pass.
- The first real run makes only expected changes.
- The second run has `changed=0`.
- The drifted file is restored and the service-level health test passes.

Cleanup

- Remove the lab package only if it was not previously required.
- Remove generated files and configuration.
- Verify the target returns to the documented baseline.

Interview Questions and Model Answers

1. What is idempotency?

Repeated execution with the same inputs converges on the same final state without unnecessary changes.

2. When is `changed_when` appropriate?

When a module or command result has a reliable signal that accurately identifies whether state changed; it should not suppress inconvenient output.

3. Why quote file modes?

YAML numeric interpretation can alter intended octal values. Quoted strings make the desired mode explicit.

4. What does the play recap not prove?

It does not prove that users can reach the service, configuration is secure, performance is acceptable, or recovery works.

Chapter Summary

- Playbooks are reviewable desired-state blueprints.
- Task names, scopes, and module choices are controls.
- Idempotency must be demonstrated with a second run.
- Validate the service, not only the recap.

Chapter 26: Variables, Precedence, Facts, Registered Data, and Filters

Ansible can load variables from many locations, which makes it flexible but creates hidden override risk. Teams should use a deliberate data model, minimize duplicate definitions, validate values, and inspect resolved host data when behavior is surprising.

Learning Objectives

- Model simple and structured variables in YAML.
- Choose variable scope and understand precedence.
- Use facts, registered results, magic variables, and filters safely.
- Protect sensitive values and avoid logging secret-bearing data.

Core Concepts

Variable data model: Lists and dictionaries should reflect the domain rather than mirror task implementation. Stable schemas simplify templates, loops, and validation.

Scope: Global, play, and host scopes determine where a value is visible. Inventory and facts are normally associated with each host.

Precedence: Ansible loads possible definitions and applies precedence rules. Extra vars have extremely high precedence; role defaults are intentionally easy to override.

Facts: The setup module gathers system information. Gather only required subsets at scale and consider fact caching confidentiality and staleness.

Registered result: register stores module return data for the current playbook run. A loop produces a results list that must be handled explicitly.

Filters and tests: Jinja filters transform data; tests evaluate conditions. Both run on the control node and require type awareness.

Data Type	Example Use	Control
Inventory variable	Region or connection metadata	One authoritative location
Role default	Safe overridable behavior	Typed/documented interface
Fact	OS family or interface data	Limit subsets and cache lifetime
Registered result	Command or module response	Inspect schema and errors
Extra var	Emergency explicit override	Restrict and audit

Reference Configuration or Command Pattern

```
# group_vars/web.yml
web_config:
  listen_port: 8080
  workers: 4
  features:
    metrics: true
    tracing: false

# tasks
- name: Gather only required facts
  ansible.builtin.setup:
    gather_subset:
      - '!all'
      - min
      - network

- name: Validate web configuration
  ansible.builtin.assert:
    that:
      - web_config.listen_port | int >= 1024
      - web_config.listen_port | int <= 65535
      - web_config.workers | int > 0
    fail_msg: Invalid web_config input

- name: Query service status
  ansible.builtin.command: systemctl is-active example
  register: service_status
  changed_when: false
  failed_when: service_status.rc not in [0, 3]
```

Production Checklist

- Each policy variable has one normal source of truth.
- Structured variables have documented schemas and defaults.
- Extra vars are treated as privileged runtime input.
- Fact gathering is limited to necessary subsets.
- Registered results are validated before nested attributes are used.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Unexpected value at runtime	Higher-precedence source overrides expected value	Use <code>ansible-inventory --host</code> , debug carefully, and trace definitions	Resolved value and source are documented
Undefined nested key	Schema differs or optional value omitted	Use defaults, validation, or explicit conditional	All supported inputs pass validation
Secret appears in output	debug, diff, failure, or verbose logging exposed value	Stop, restrict logs, rotate secret, add <code>no_log</code> selectively	Secret scan passes and credential is rotated

Symptom	Likely Cause	Response	Validation
Fact-based task chooses wrong branch	Stale cache or wrong fact assumption	Refresh subset and inspect exact fact	Condition uses verified current fact

Hands-on Lab 26: Build and Validate a Structured Variable Model

Prerequisites

- Ansible project and disposable target.
- A group_vars directory.
- No real secrets.

Implementation

1. Create a structured web_config dictionary with required and optional values.
2. Add an assert task that validates port, worker count, and allowed environment.
3. Gather minimal facts and display only non-sensitive selected fields.
4. Register a harmless command result and use rc/stdout in a conditional assertion.
5. Override one value from inventory, play vars, and extra vars to observe precedence; then redesign to a single normal source.
6. Use filters to normalize a list of package names and remove duplicates.

Validation Evidence

- Valid input passes and invalid input fails before change.
- Fact gathering returns only intended subsets.
- Registered result handling covers success and expected inactive status.
- The final model has no ambiguous duplicate variable definitions.

Cleanup

- Remove temporary override files.
- Delete evidence containing host-specific facts not needed for the portfolio.
- Retain sanitized schema and validation examples.

Interview Questions and Model Answers

1. What is the safest way to handle variable precedence?

Avoid relying on complex precedence. Define each policy value once in a documented scope and use explicit role parameters or approved runtime overrides.

2. Are registered variables persistent?

No. They are stored in memory for the current playbook run and associated with the host.

3. Where do Jinja filters execute?

On the control node, because templating and data manipulation happen before module arguments are sent to the target.

4. Does no_log encrypt a secret?

No. It suppresses normal task output. The value still needs secure storage, transport, access control, and rotation.

Chapter Summary

- Variables are an API and need schemas.

- Minimize duplicate definitions and precedence dependence.
- Facts and registered data require validation.
- Secret handling is broader than output suppression.

Chapter 27: Conditionals, Loops, Tags, Delegation, and Run Control

Execution-control features can reduce duplication and support heterogeneous fleets, but excessive branching turns a playbook into hard-to-audit procedural code. Use clear data structures, limited conditions, deliberate tags, and explicit delegation boundaries.

Learning Objectives

- Use when conditions and Jinja tests correctly.
- Build readable loops with loop_control and registered results.
- Apply tags without creating unsafe partial configurations.
- Use run_once, delegate_to, serial, throttle, and strategy deliberately.

Core Concepts

Conditional: A when expression decides whether a task applies to a host. Prefer selecting data or files by platform over deeply nested task logic.

Loop: Repeats one task over structured data. Stable labels and explicit loop variables make output and nested reuse understandable.

Tags: Select subsets of tasks at preprocessing time. Tags are operational interfaces and require documentation and dependency awareness.

Delegation: delegate_to executes a task on another host, often localhost, a load balancer, or an API endpoint. Variable context must be understood.

run_once: Executes a task for one host in the current batch; the chosen host and variable context can surprise operators.

Rollout control: serial limits hosts per batch, throttle limits concurrent instances of a task, and strategy affects host-task scheduling.

Feature	Use Case	Primary Risk
when	OS or feature-specific task	Hidden branching
loop	Repeated homogeneous resources	Unreadable nested data
tags	Operational subset	Partial dependency execution
delegate_to	External control-plane action	Wrong variable context
serial	Rolling update	Capacity and failure math

Reference Configuration or Command Pattern

```
- name: Roll out configuration safely
hosts: web
become: true
serial: 1
max_fail_percentage: 0

tasks:
  - name: Validate supported operating system
    ansible.builtin.assert:
      that: ansible_facts.os_family in ['Debian', 'RedHat']

  - name: Configure application users
    ansible.builtin.user:
      name: "{{ app_user.name }}"
      groups: "{{ app_user.groups | join(',') }}"
      state: present
    loop: "{{ app_users }}"
    loop_control:
      loop_var: app_user
      label: "{{ app_user.name }}"
    tags: [accounts]

  - name: Remove host from load balancer
    ansible.builtin.debug:
      msg: "Drain {{ inventory_hostname }}"
    delegate_to: localhost
    tags: [deployment]
```

Production Checklist

- Conditions are based on validated variables or current facts.
- Loops use named loop variables and concise labels.
- Tags are documented with prerequisites and safe combinations.
- Delegated tasks state where credentials and variables come from.
- Serial rollout preserves required service capacity and has stop criteria.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Task skipped unexpectedly	Condition type, undefined value, or fact mismatch	Debug only necessary values and use type tests	Condition evaluates correctly for each supported host
Nested loop overwrites item	Default loop variable collision	Use loop_control.loop_var	Each level has distinct variable
Tagged run breaks service	Skipped prerequisite or handler	Document tag contract; use always/never carefully; test subsets	Selected tag path is independently valid
run_once uses wrong host data	First host differs from intended authority	Delegate explicitly and use hostvars with named host	Result is sourced from intended system

Hands-on Lab 27: Create a Controlled Rolling Configuration Workflow

Prerequisites

- At least two disposable web host aliases.
- A harmless configuration file or service simulation.
- Inventory facts available.

Implementation

1. Create a play with serial: 1 and max_fail_percentage: 0.
2. Use an assert condition to reject unsupported OS families.
3. Loop over a structured application-user list with loop_var and label.
4. Add deployment and verification tags and document safe invocation combinations.
5. Delegate a simulated load-balancer drain and rejoin step to localhost.
6. Run --list-tasks and --list-tags, then execute against one canary and the full group.

Validation Evidence

- Tasks execute one host at a time.
- The unsupported-host assertion fails before modification.
- Loop output identifies users clearly.
- Tagged paths include required verification and do not leave a host drained.

Cleanup

- Remove created lab users or files.
- Verify all simulated hosts are rejoined.
- Retain rollout and tag-contract documentation.

Interview Questions and Model Answers

1. Why can tags be dangerous?

They allow operators to skip tasks that another task assumes have run, creating partial and invalid configurations.

2. What does serial control?

It divides the host set into batches for a play, enabling rolling operations and bounded failure impact.

3. When should delegate_to be used?

When an action belongs on another control point, such as an API host, load balancer, monitoring system, or localhost.

4. Why avoid deeply nested conditions?

They make behavior difficult to review and test. Prefer clear data selection, separate task files, roles, or inventory groups.

Chapter Summary

- Execution controls are part of the automation interface.
- Keep conditions and loops readable.
- Treat tags as tested operational contracts.
- Design rolling capacity and delegated trust explicitly.

Chapter 28: Jinja2 Templates and Configuration-File Engineering

Configuration files are high-consequence interfaces between automation and services. A template must render deterministic content, validate before replacement, preserve secure ownership and mode, avoid volatile values, and trigger only the minimum required service action.

Learning Objectives

- Render Jinja2 templates from structured variables and facts.
- Use filters, tests, defaults, and whitespace control.
- Validate candidate configuration before installation.
- Design templates for idempotency, security, and maintainability.

Core Concepts

Control-node rendering: Jinja expressions are evaluated on the control node using host-scoped data, then the rendered content is transferred to the target.

Template contract: The template expects a documented variable schema. Assertions should fail early when required keys or invariants are absent.

Determinism: Current timestamps, random order, and unstable queries cause needless changes. Sort collections and avoid volatile data unless semantically required.

Validation: The template module validate option can run a safe command against a temporary candidate before replacing the destination.

Whitespace: Jinja block trimming and indentation affect configuration syntax. Small visual changes can be operationally significant.

Sensitive rendering: Rendered files, diffs, backups, and failure output can contain secrets. Use restrictive modes, disable diff where necessary, and source secrets securely.

Template Concern	Control	Evidence
Schema	assert before template	Invalid fixture fails
Ordering	sort/dictsort	Second run unchanged
Syntax	validate command	Bad candidate rejected
Permissions	owner/group/mode	stat verification
Secrets	no_log/diff control/external source	Artifact scan

Reference Configuration or Command Pattern

```
# templates/example.conf.j2
# Managed by Ansible. Manual changes will be overwritten.
listen_port = {{ web_config.listen_port | int }}
workers = {{ web_config.workers | int }}

{% for backend in web_config.backends | sort(attribute='name') %}
backend {{ backend.name }} {
  address = {{ backend.address }}
  weight = {{ backend.weight | default(100) | int }}
}
{% endfor %}

# task
- name: Install validated service configuration
  ansible.builtin.template:
    src: example.conf.j2
    dest: /etc/example/example.conf
    owner: root
    group: root
    mode: '0640'
    validate: /usr/local/bin/example-check --config %s
    backup: true
  notify: Reload example service
```

Production Checklist

- Template inputs are validated and documented.
- Output ordering is deterministic.
- Candidate syntax is checked before replacement.
- Ownership, group, mode, and backup policy are explicit.
- Secret-bearing templates suppress unsafe diff and protect backups.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Template changes every run	Timestamp, unordered data, or unstable facts	Remove volatility and sort collections	Second run changed=0
Service rejects new config	No candidate validation or incomplete test	Add validate command and service-specific test	Invalid template is rejected before replacement
Undefined variable	Schema mismatch or wrong scope	Assert required fields and inspect resolved host vars	Supported fixtures render successfully
Secret exposed in diff	Diff enabled for sensitive destination	Disable diff for task, restrict artifacts, rotate if exposed	Logs and backups contain no exposed secret

Hands-on Lab 28: Render and Validate a Service Configuration

Prerequisites

- Disposable target.

- A simple validation script that accepts or rejects the candidate file.
- Structured web_config variables.

Implementation

1. Create a Jinja2 template with sorted backend entries and defaults.
2. Add assertions for port range, worker count, backend names, and addresses.
3. Write a validation script and configure template validate with %s.
4. Render a valid configuration and inspect owner, group, mode, and content.
5. Introduce an invalid value and prove the destination is not replaced.
6. Run twice with valid data and verify changed=0; then alter one semantic input and inspect --diff.

Validation Evidence

- Valid content installs and passes the validator.
- Invalid content fails before destination replacement.
- File permissions match policy.
- Stable inputs produce an unchanged second run.

Cleanup

- Restore or remove the lab configuration.
- Remove validation helper if created only for the lab.
- Securely delete sensitive backup or diff artifacts.

Interview Questions and Model Answers

1. Where is Jinja2 evaluated?

On the Ansible control node before the rendered content is sent to the target.

2. Why sort data in templates?

Deterministic ordering prevents meaningless diffs and unnecessary service reloads.

3. What does validate do?

It runs a command against a temporary rendered file before Ansible replaces the destination; %s is substituted with the temporary path.

4. Should every template include a timestamp?

No. A changing timestamp destroys idempotency unless the timestamp itself is a required part of the managed configuration.

Chapter Summary

- Templates are configuration APIs, not text substitution tricks.
- Validate schema and candidate syntax.
- Keep rendering deterministic.
- Protect secret-bearing outputs and backups.

Chapter 29: Handlers, Blocks, Error Handling, Check Mode, and Diff Mode

Production automation must control side effects and failure paths. Handlers prevent unnecessary restarts, blocks create transactional structure, and check/diff modes improve review; none of them replace service-aware validation, rollback, or incident procedures.

Learning Objectives

- Use notify, handlers, listen topics, and flush_handlers correctly.
- Group tasks with block, rescue, and always.
- Define honest failure and change conditions.
- Understand the limits of check mode and diff mode.

Core Concepts

Handler: A task that runs only when notified by a changed task. Repeated notifications normally coalesce into one execution per handler flush point.

Handler order: Handlers run in definition/insertion order, not notification order. Name collisions and role insertion order require discipline.

flush_handlers: Forces currently notified handlers to run before the normal end-of-section flush. Use only when later tasks require the new service state.

Block/rescue/always: A block groups tasks and applies directives. rescue handles task failure; always runs cleanup or evidence collection regardless of block outcome.

Check mode: Simulates supported modules without change. Unsupported tasks may skip, and registered data or external side effects can make the prediction incomplete.

Diff mode: Shows before/after details for supported modules. It can leak sensitive content and should be disabled selectively.

Mechanism	Purpose	Limitation
Handler	Run side effect only after change	May not run after later failure unless forced
Block/rescue	Structured recovery	Not a full transaction rollback
Check mode	Predict supported changes	Not all modules or dependencies simulate
Diff mode	Review content change	Can expose secrets
failed_when	Define domain failure	Can hide real failure if careless

Reference Configuration or Command Pattern

```
- name: Update service safely
block:
  - name: Render validated configuration
    ansible.builtin.template:
      src: example.conf.j2
      dest: /etc/example/example.conf
      mode: '0640'
      validate: /usr/local/bin/example-check --config %s
    notify: Reload example service

  - name: Flush configuration reload before health test
    ansible.builtin.meta: flush_handlers

  - name: Verify local health endpoint
    ansible.builtin.uri:
      url: http://127.0.0.1:8080/health
      status_code: 200
    register: health
    retries: 5
    delay: 2
    until: health.status == 200

rescue:
  - name: Restore known-good configuration
    ansible.builtin.copy:
      src: /var/backups/example.conf.good
      dest: /etc/example/example.conf
      remote_src: true
      mode: '0640'
    notify: Reload example service

  - name: Stop the play after recovery attempt
    ansible.builtin.fail:
      msg: Configuration deployment failed; recovery was attempted

always:
  - name: Record final service status
    ansible.builtin.command: systemctl is-active example
    register: final_status
    changed_when: false
    failed_when: false

handlers:
  - name: Reload example service
    ansible.builtin.systemd_service:
      name: example
      state: reloaded
```

Production Checklist

- Handlers are named uniquely and perform the minimum safe action.

- Configuration validation occurs before a reload or restart.
- Recovery blocks preserve the original failure and record evidence.
- Check-mode gaps are documented per module and task.
- Sensitive tasks disable diff and protect backup content.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Handler not executed	Not notified, task unchanged, name mismatch, or host failed first	Inspect changed status and notification names; consider <code>force_handlers</code> carefully	Handler executes only on intended change
Health test runs before reload	Handler deferred until play section end	Use <code>flush_handlers</code> at a justified boundary	Health test observes new config
Check mode says no change but real run changes	Unsupported task or registered-data dependency	Review module attributes and test on disposable canary	Preview limitations documented
Rescue masks root cause	Recovery output replaces original context	Capture original result and fail with clear incident message	Evidence contains initial and recovery outcomes

Hands-on Lab 29: Implement a Validated Change with Handler and Recovery

Prerequisites

- Disposable service or a simulated service script.
- Known-good configuration backup.
- A health-check endpoint or validation command.

Implementation

1. Create a template task that notifies a reload handler only on change.
2. Flush handlers before a health-check task.
3. Wrap deployment and health validation in a block.
4. Add rescue tasks that restore known-good configuration and notify the handler.
5. Add always tasks that record final status without hiding failure.
6. Run syntax, check/diff, successful deployment, idempotency, and forced-failure scenarios.

Validation Evidence

- Successful change reloads exactly once.
- Unchanged run does not reload.
- Invalid candidate is rejected before replacement.
- Forced health failure triggers recovery and leaves clear failed status and evidence.

Cleanup

- Restore original service configuration.
- Remove temporary backups after evidence is sanitized.
- Confirm the service is healthy and no handler remains pending.

Interview Questions and Model Answers

1. When do handlers run?

Normally after `pre_tasks`, the main roles/tasks section, and `post_tasks`, with notifications coalesced; explicit `flush_handlers` can run them earlier.

2. What is the purpose of rescue?

It defines recovery tasks after a block task fails, but it does not automatically roll back all prior external side effects.

3. Is check mode a guarantee?

No. It is a simulation whose completeness depends on module support, registered values, external APIs, and task design.

4. Why can diff mode be risky?

It may reveal secrets or sensitive configuration in terminal output and stored logs.

Chapter Summary

- Handlers minimize side effects by reacting to real change.
- Blocks structure recovery but do not create transactions.
- Check and diff improve review within known limits.
- Failure evidence and service health remain mandatory.

Chapter 30: Vault, Secret Handling, Quality Gates, and Configuration Capstone

Secrets and quality controls determine whether Ansible automation is safe to operate beyond a lab. Vault protects data at rest, but production secret management also requires identity, access control, rotation, runtime retrieval, output suppression, and artifact hygiene. A mature workflow validates syntax, lint, preview, idempotency, and service behavior before broad rollout.

Learning Objectives

- Encrypt variables and files with Ansible Vault and multiple Vault identities.
- Distinguish encryption at rest from runtime secret-management controls.
- Build a local quality sequence with syntax, lint, check, diff, and idempotency.
- Integrate inventory, variables, templates, handlers, recovery, and evidence in a capstone.

Core Concepts

Ansible Vault: Encrypts variables or files so sensitive content is not stored as plaintext. Decryption occurs for execution, so runtime exposure controls still matter.

Vault identity: A label and password source used to select different decryption credentials for environments or trust domains.

Secret reference: Where possible, retrieve a secret at runtime from an approved external manager rather than storing the secret value in the repository.

no_log: Suppresses normal task output but does not encrypt memory, files, state on the target, or external logs.

Quality gate: A repeatable sequence such as YAML lint, ansible-lint, syntax check, inventory validation, check/diff, canary, idempotency, and acceptance tests.

Capstone evidence: Another engineer should be able to reproduce targeting, inputs, predicted changes, actual changes, second-run stability, service validation, recovery, and cleanup.

Control	Protects Against	Does Not Replace
Vault encryption	Plaintext repository secret	Runtime access and rotation
no_log	Normal task output exposure	Secure target files and external logs
Syntax/lint	Structural and common quality defects	Integration testing
Check/diff	Many predicted file/resource changes	Real canary execution
Idempotency test	Unnecessary repeated change	Functional acceptance

Reference Configuration or Command Pattern

```
# Encrypt a variable file with an identity
ansible-vault encrypt --vault-id dev@prompt group_vars/dev/vault.yml
ansible-vault view --vault-id dev@prompt group_vars/dev/vault.yml

# Quality sequence
ansible-inventory -i inventory --graph
ansible-playbook playbooks/site.yml --syntax-check
ansible-playbook playbooks/site.yml --list-hosts
ansible-playbook playbooks/site.yml --check --diff --limit canary
ansible-playbook playbooks/site.yml --limit canary
ansible-playbook playbooks/site.yml --limit canary # idempotency

# Example secret-bearing task
- name: Install application credential
  ansible.builtin.template:
    src: credential.conf.j2
    dest: /etc/example/credential.conf
    owner: root
    group: example
    mode: '0640'
  no_log: true
  diff: false
  notify: Reload example service
```

Production Checklist

- Vault password sources are outside the repository and access-controlled.
- Different environments or trust domains use separate identities.
- Secret-bearing tasks disable unsafe output and install restrictive target permissions.
- Quality gates run from pinned dependencies.
- Canary, idempotency, functional validation, and recovery are required before rollout.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Vault file cannot decrypt	Wrong identity, password source, or corrupted file	Verify label and secure password source; restore version if corrupted	vault view succeeds for authorized operator
Secret appears despite no_log	Template diff, callback, target log, backup, or downstream command exposed it	Contain, rotate, remove artifacts, and map every exposure path	Secret scan and rotation complete
Check mode passes but canary fails	Unsupported simulation or runtime dependency	Inspect module support and canary evidence; add integration test	Canary and health validation pass
Second run changes configuration	Volatile template, command semantics, or handler loop	Inspect diff and changed result; make input deterministic	Idempotency run changed=0

Hands-on Lab 30: Ansible Foundation Capstone: Secure Web Configuration Platform

Prerequisites

- Completed Chapters 21-29.
- Two disposable Linux targets or containers.
- A Git repository and isolated Ansible environment.

Implementation

1. Create dev and production-like inventory groups with a canary host and explicit connection variables.
2. Model a structured web_service variable schema and encrypt one simulated credential with a named Vault identity.
3. Create a playbook that validates variables, installs a package, creates users/directories, renders validated configuration, and manages a service.
4. Use handlers, a serial canary rollout, delegated drain/rejoin simulation, block/rescue recovery, and a functional health test.
5. Run inventory graph, syntax, list-hosts, lint if available, check/diff, canary apply, idempotency, and full-group rollout.
6. Introduce manual drift and prove check/diff detects it; reconcile through reviewed automation.
7. Force an invalid candidate or health failure and demonstrate recovery evidence.
8. Write README and RUNBOOK covering normal run, secrets, targeting, rollback, failed host, handler failure, and cleanup.

Validation Evidence

- The resolved target set is explicit and bounded.
- Vault-protected data is never exposed in repository, diff, or logs.
- The first run converges, the second run changes nothing, and health tests pass.
- Drift is detected and reconciled.
- Failure simulation executes the documented recovery path.

Cleanup

- Remove lab packages, users, files, and services where safe.
- Delete temporary decrypted files, Vault password files, diffs, and secret-bearing logs.
- Revoke temporary SSH keys and credentials.
- Retain only sanitized code, diagrams, runbook, test results, and lessons learned.

Interview Questions and Model Answers

1. What does Ansible Vault protect?

It protects selected variables or files at rest. It does not replace runtime identity, least privilege, rotation, secure target storage, or log controls.

2. What is a production Ansible quality gate?

Pinned execution environment, inventory validation, syntax and lint, check/diff where supported, canary execution, idempotency, functional tests, security checks, approval, and evidence.

3. How do Terraform and Ansible complement each other?

Terraform manages infrastructure resource lifecycle and state. Ansible converges configuration and orchestrates procedural operations. Clear ownership prevents both tools from fighting over the same attribute.

4. What proves this capstone is production-ready?

Bounded targeting, reproducible dependencies, secure identity and secrets, deterministic templates, idempotency, canary rollout, service validation, recovery tests, cleanup, and a runbook another engineer can execute.

Chapter Summary

- Vault is one layer of a complete secret-management system.
- Automated quality gates reduce but do not eliminate change risk.
- Canary, idempotency, service validation, and recovery produce operational confidence.
- The capstone integrates the complete Ansible foundation workflow.

Part C Ansible Command Reference

Run every command from the intended project, inventory, configuration, and credential context. Use `--list-hosts` or `--limit` before broad changes, and protect verbose output because it may contain infrastructure or secret data.

Purpose	Command	Operational Note
Version	<code>ansible --version</code>	Record executable, core, Python, Jinja, and collection paths.
Configuration source	<code>ansible-config view</code>	Shows the active configuration content.
Changed settings	<code>ansible-config dump --only-changed</code>	Confirms non-default behavior and source.
Module documentation	<code>ansible-doc ansible.builtin.copy</code>	Review arguments, return values, attributes, and examples.
Inventory graph	<code>ansible-inventory -i inventory --graph</code>	Inspect groups and host membership.
Resolved host vars	<code>ansible-inventory -i inventory --host HOST</code>	May expose sensitive inventory data; protect output.
Target preview	<code>ansible PATTERN -i inventory --list-hosts</code>	Mandatory before high-impact ad hoc operations.
Connectivity	<code>ansible PATTERN -m ansible.builtin.ping</code>	Tests connection and Python module execution.
Facts	<code>ansible HOST -m ansible.builtin.setup -a "filter=ansible_distribution*"</code>	Gather only needed fields.
Syntax	<code>ansible-playbook site.yml --syntax-check</code>	Parses playbook structure; does not contact all targets.
List hosts	<code>ansible-playbook site.yml --list-hosts</code>	Confirms the play target set.
List tasks	<code>ansible-playbook site.yml --list-tasks</code>	Review static task expansion and tags.
Preview	<code>ansible-playbook site.yml --check --diff</code>	Simulation is limited by module support and data dependencies.
Canary	<code>ansible-playbook site.yml --limit web01</code>	Use a representative low-risk target.
Start at task	<code>ansible-playbook site.yml --start-at-task "Task name"</code>	Useful for diagnosis; respect prerequisites.
Step mode	<code>ansible-playbook site.yml --step</code>	Interactive task confirmation for controlled troubleshooting.
Tags	<code>ansible-playbook site.yml --tags deployment</code>	Run only tested and documented tag paths.
Vault encrypt	<code>ansible-vault encrypt --vault-id dev@prompt FILE</code>	Password source must remain outside repository.

Purpose	Command	Operational Note
Vault view	<code>ansible-vault view --vault-id dev@prompt FILE</code>	Decrypted output is sensitive.
Collection install	<code>ansible-galaxy collection install -r requirements.yml</code>	Pin and review collection versions.
Collection list	<code>ansible-galaxy collection list</code>	Capture content inventory for evidence.

Ansible Foundation Production-Readiness Checklist

Control Node and Dependencies

- ☐ Ansible, Python, collections, and system dependencies are pinned.
- ☐ The active `ansible.cfg` and plugin paths are verified.
- ☐ `requirements` files and execution-environment build inputs are reviewed.
- ☐ Credentials are supplied at runtime through approved mechanisms.

Inventory and Targeting

- ☐ Inventory has an owner and authoritative source.
- ☐ Host identity, groups, and variable placement follow a standard.
- ☐ Production jobs record resolved hosts and limits.
- ☐ Dynamic inventory filtering, caching, and stale-data behavior are tested.

Connectivity and Privilege

- ☐ Host-key checking and known-host management are enabled.
- ☐ Automation accounts use least privilege.
- ☐ Become rules are scoped and audited.
- ☐ Interpreter and bastion requirements are documented.

Playbook Quality

- ☐ FQCN modules and descriptive task names are used.
- ☐ Variables have schemas and assertions.
- ☐ Templates are deterministic and validated before replacement.
- ☐ Command/shell tasks have guards and honest change/failure rules.

Change Safety

- ☐ Syntax, target, check, and diff evidence is captured.
- ☐ Canary and serial rollout rules preserve capacity.
- ☐ Second-run idempotency is required.
- ☐ Functional, security, and recovery validation follows execution.

Secrets and Evidence

- ☐ Vault identities or external secret references follow trust boundaries.
- ☐ `no_log` and `diff` controls are applied to secret-bearing tasks.
- ☐ Logs, backups, rendered files, and facts are access-controlled.
- ☐ Secret rotation and exposure response are documented.

Ten Additional Ansible Foundation Projects

1. Inventory Governance Toolkit

Validate naming, required groups, variable schemas, duplicate endpoints, and approved target patterns.

- Deliverables: sanitized repository, README, inventory model, tests, runbook, evidence, cleanup proof, risks, and lessons learned.

2. SSH Readiness Auditor

Test DNS, host keys, account access, Python, become, and bastion paths with sanitized reports.

- Deliverables: sanitized repository, README, inventory model, tests, runbook, evidence, cleanup proof, risks, and lessons learned.

3. Ad Hoc Safety Wrapper

Require inventory graph, host-list confirmation, canary, fork limit, and evidence before one-time operations.

- Deliverables: sanitized repository, README, inventory model, tests, runbook, evidence, cleanup proof, risks, and lessons learned.

4. Linux Baseline Playbook

Converge users, packages, time, logging, permissions, and service health with idempotency tests.

- Deliverables: sanitized repository, README, inventory model, tests, runbook, evidence, cleanup proof, risks, and lessons learned.

5. Variable Contract Library

Create documented structured schemas, assertions, valid fixtures, and negative fixtures for common services.

- Deliverables: sanitized repository, README, inventory model, tests, runbook, evidence, cleanup proof, risks, and lessons learned.

6. Deterministic Template Pack

Build validated templates for web, systemd, logging, and application configuration with secret-safe diff controls.

- Deliverables: sanitized repository, README, inventory model, tests, runbook, evidence, cleanup proof, risks, and lessons learned.

7. Handler and Recovery Simulator

Inject configuration, reload, and health failures and prove block/rescue/always behavior.

- Deliverables: sanitized repository, README, inventory model, tests, runbook, evidence, cleanup proof, risks, and lessons learned.

8. Check-Mode Compatibility Matrix

Test selected modules and document where preview is complete, partial, skipped, or misleading.

- Deliverables: sanitized repository, README, inventory model, tests, runbook, evidence, cleanup proof, risks, and lessons learned.

9. Vault Identity Demonstration

Separate dev and production-like identities, rotate passwords, rekey files, and audit access.

- Deliverables: sanitized repository, README, inventory model, tests, runbook, evidence, cleanup proof, risks, and lessons learned.

10. Ansible Foundation CI Pipeline

Automate dependency installation, inventory validation, syntax, lint, check, idempotency, and container acceptance tests.

- Deliverables: sanitized repository, README, inventory model, tests, runbook, evidence, cleanup proof, risks, and lessons learned.

Senior Ansible Foundation Interview Drill Bank

1. How does Ansible execute a Linux module?

The control node renders arguments, connects through the selected plugin, transfers or invokes module code, executes it through the target interpreter, receives structured results, and removes temporary artifacts according to configuration.

2. How do you prevent running against the wrong hosts?

Use authoritative inventory, explicit project and inventory paths, inventory graph and --list-hosts, restrictive patterns and --limit, canary execution, approvals, and job evidence containing the resolved host set.

3. What is Ansible idempotency?

The same desired-state automation with unchanged inputs produces the same final state and no unnecessary changes on subsequent runs.

4. Why can variable precedence be dangerous?

The same name can be defined in many scopes and the winning value may be non-obvious. Mature designs define policy values once and use explicit interfaces.

5. What are the limits of check mode?

Only supporting modules simulate; external side effects, registered-data dependencies, commands, and service behavior may not be represented. It is preview evidence, not a guarantee.

6. How should templates be made safe?

Validate input schemas, render deterministic ordering, use candidate validation, set ownership and mode, suppress secret-bearing diffs, notify minimal handlers, and perform service health tests.

7. Why use handlers?

They defer and coalesce service side effects so reloads or restarts occur only when notifying tasks actually change state.

8. What does Ansible Vault not solve?

Runtime authorization, secret rotation, secure delivery, target-file protection, process exposure, external logs, backups, or incident response.

9. When should command or shell be used?

Only when no suitable module exists, with explicit creates/removes or changed_when/failed_when, safe quoting, bounded inputs, and validation.

10. How do Terraform and Ansible avoid conflict?

Assign ownership by lifecycle and attribute: Terraform owns infrastructure resources and state; Ansible owns approved configuration domains. Outputs and inventory contracts connect them without both tools managing the same setting.

Glossary

Term	Definition
Ad hoc command	A single-task Ansible invocation executed with the ansible CLI rather than a playbook.
Ansible Vault	A feature that encrypts selected variables or files at rest.
Become	Ansible privilege-escalation framework used to execute with another effective identity.
Check mode	A simulation mode that predicts changes for supporting modules without modifying targets.
Collection	A distribution format for modules, plugins, roles, playbooks, and documentation under a namespace.
Control node	The system where Ansible commands, plugins, templating, and orchestration execute.
Diff mode	A mode that displays before-and-after content for supporting modules.
Fact	Host data discovered by the setup module or other fact providers.
FQCN	Fully Qualified Collection Name, such as <code>ansible.builtin.template</code> .
Handler	A task that runs after being notified by a changed task.
Host pattern	An expression that selects inventory hosts and groups.
Idempotency	The property that repeated runs with unchanged inputs produce no unnecessary changes.
Inventory	Sources that define managed hosts, groups, and associated variables.
Jinja2	The templating engine used by Ansible for expressions and rendered files.
Managed node	A target system or service controlled by Ansible.
Play	A mapping of a host pattern to tasks, variables, privilege, and execution behavior.
Registered variable	A host-scoped in-memory variable containing a task result for the current playbook run.
Rescue	Tasks executed after a task fails within an Ansible block.
Role	A reusable Ansible content structure containing tasks, handlers,

Term	Definition
	defaults, vars, files, templates, and metadata.
Vault identity	A label and password source used to select decryption credentials for encrypted content.

Official Ansible Documentation Index

Reference	Official URL
Ansible documentation	https://docs.ansible.com/projects/ansible/latest/
Installation guide	https://docs.ansible.com/projects/ansible/latest/installation_guide/
Inventory guide	https://docs.ansible.com/projects/ansible/latest/inventory_guide/
Inventory introduction	https://docs.ansible.com/projects/ansible/latest/inventory_guide/intro_inventory.html
Command-line tools	https://docs.ansible.com/projects/ansible/latest/command_guide/
Playbook guide	https://docs.ansible.com/projects/ansible/latest/playbook_guide/
Playbook introduction	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_intro.html
Variables	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_variables.html
Facts and magic variables	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_vars_facts.html
Conditionals	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_conditionals.html
Loops	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_loops.html
Tags	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_tags.html
Jinja2 templating	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_templating.html
Filters	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_filters.html
Handlers	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_handlers.html

Reference	Official URL
Blocks and error handling	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_blocks.html
Check and diff mode	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_checkmode.html
Vault guide	https://docs.ansible.com/projects/ansible/latest/vault_guide/
Collections guide	https://docs.ansible.com/projects/ansible/latest/collections_guide/
Built-in collection index	https://docs.ansible.com/projects/ansible/latest/collections/ansible/builtin/
Configuration settings	https://docs.ansible.com/projects/ansible/latest/reference_appendices/config.html
Precedence rules	https://docs.ansible.com/projects/ansible/latest/reference_appendices/general_precedence.html
Release and maintenance	https://docs.ansible.com/projects/ansible/latest/reference_appendices/release_and_maintenance.html

Ansible, ansible-core, collections, Python support, templating behavior, module attributes, and platform requirements evolve. Verify the current stable documentation, porting guides, collection changelogs, and supported execution environment before production rollout.

Volume 9C Completion

Part C completes Ansible foundations and configuration automation. The next section moves to reusable roles and collections, execution environments, enterprise controllers, network and Windows automation, testing, and scale operations.

Next part: Volume 9D - Advanced Ansible Engineering, Roles, Collections, Execution Environments, AWX/Automation Platform, Windows, Network Automation, Testing, and Enterprise Operations.